KCPC Week 5 Solutions
Graphs II: Dijkstra and 0–1 BFS

These notes retain **all 15 original problems**, in the original order: live walkthroughs 1–3, core practice 4–11, and extended homework 12–15. Every solution includes its model, proof, complexity and common mistakes. Problems 1–11 have Python 3 and C++17 implementations; homework has C++17 implementations.

Provenance. Problem statements belong to the linked platforms. All implementations below were written independently for this sheet; no external solution code is reproduced. The request to reuse Peng Cao's work cannot be actioned without an identifiable source; no attribution or reuse is claimed. Formatting follows the KCPC Week 3 solutions template.

Running the listings. GFG/LeetCode listings are independent judge classes: submit just one listing, including its imports or headers, without a `main`. AtCoder/Codeforces listings are complete stdin/stdout programs. Listings labelled `p09-p11-python` and `p09-p11-cpp` are complete shared Euler programs, with usage below. Every listing has a stable `lst:...` label in the source, making extraction unambiguous.

Reusable reasoning. For non-negative additive weights, Dijkstra finalises a vertex when its smallest current distance is popped: if a cheaper path existed, its first unfinalised vertex would already have a no-larger heap key, a contradiction. Every relaxation describes a real path, so final distances are optimal. Lazy heaps insert a new entry instead of deleting an old one; skip entries whose key no longer equals the stored label. With N vertices and M directed adjacency entries, there are $O(M+1)$ insertions: time $O(N + M\log(M+2))$, auxiliary heap space $O(M+1)$, and total graph/label/heap space $O(N+M)$. These are *not* $O(N)$-heap implementations.

For weights in $\{0,1\}$, a deque maintains nondecreasing live distances: a zero-cost relaxation goes to the front, a one-cost relaxation to the back. This is Dijkstra with two adjacent distance buckets, hence correct in $O(N+M)$ time and $O(N+M)$ space including stale entries. Our grid graphs have constant degree, so this becomes linear in the number of cells. Ordinary FIFO BFS is sufficient only when every edge has the same cost.

I. Integrated examples

1 GFG — Dijkstra Algorithm

Original problem / current judge page

The old sheet says "Adjacency Matrix", but the **current interface is an edge list**: `dijkstra(V, edges, src)`, with each row `[u,v,w]`. This was checked against the platform's current statement and Python/C++ starter code. The graph is undirected, connected and non-negatively weighted; the statement guarantees that shortest distances fit in a signed 32-bit integer.

Key idea: Build both adjacency directions, then run min-heap Dijkstra from `src`.

Correctness: Initially only the empty source path is known. Relaxing $u \to v$ with $d[u]+w$ considers a valid extension. The non-negative-weight finalisation argument above proves every returned

distance is shortest; connectivity makes every distance finite.

Time and space: For V vertices and E undirected edges: $O(V + E \log(E+2))$ time and $O(V+E)$ total auxiliary space, including the adjacency list and an $O(E+1)$ lazy heap.

Pitfalls: Do not treat `edges[u]` as a neighbour list or add only one direction. Zero weights are legal. C++ uses 64-bit intermediate sums, then converts to the required `vector<int>` under the judge's guarantee.

Listing 1: P01 — Python 3

```python
from heapq import heappop, heappush

class Solution:
    def dijkstra(self, V: int, edges: list[list[int]],
                 src: int) -> list[int]:
        adj = [[] for _ in range(V)]
        for u, v, w in edges:
            adj[u].append((v, w))
            adj[v].append((u, w))
        dist = [float('inf')] * V
        dist[src] = 0
        heap = [(0, src)]
        while heap:
            d, u = heappop(heap)
            if d != dist[u]:
                continue
            for v, w in adj[u]:
                nd = d + w
                if nd < dist[v]:
                    dist[v] = nd
                    heappush(heap, (nd, v))
        return dist
```

Listing 2: P01 — C++17

```cpp
#include <functional>
#include <queue>
#include <utility>
#include <vector>
using namespace std;
class Solution {
public:
    vector<int> dijkstra(int V, vector<vector<int>>& edges,
                         int src) {
        using ll = long long;
        const ll INF = (1LL << 62);
        vector<vector<pair<int, int>>> adj(V);
        for (const auto& e : edges) {
            adj[e[0]].push_back({e[1], e[2]});
            adj[e[1]].push_back({e[0], e[2]});
        }
        vector<ll> dist(V, INF);
        using State = pair<ll, int>;
        priority_queue<State, vector<State>, greater<State>> pq;
        dist[src] = 0;
        pq.push({0, src});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d != dist[u]) continue;
            for (auto [v, w] : adj[u]) {
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    pq.push({dist[v], v});
                }
```

```
            }
        }
        return vector<int>(dist.begin(), dist.end());
    }
};
```

2 LeetCode 743 — Network Delay Time

Problem statement

Key idea: The time at which a vertex receives the signal is its shortest distance from k. The entire network receives it at the maximum of these distances.

Correctness: A directed path gives exactly its sum of transmission times. Dijkstra therefore finds each earliest arrival. If any vertex is unreachable, no finite answer exists; otherwise the latest earliest arrival is both necessary and sufficient.

Time and space: With n vertices and m directed edges, $O(n + m \log(m + 2))$ time and $O(n + m)$ space, including an $O(m + 1)$ heap.

Pitfalls: Edges are directed. Labels are 1-based; index zero is not a vertex. Take the maximum, not the sum, and return -1 for an unreachable vertex.

Listing 3: P02 — Python 3

```
from heapq import heappop, heappush
from typing import List

class Solution:
    def networkDelayTime(self, times: List[List[int]],
                         n: int, k: int) -> int:
        adj = [[] for _ in range(n + 1)]
        for u, v, w in times:
            adj[u].append((v, w))
        dist = [float('inf')] * (n + 1)
        dist[k] = 0
        heap = [(0, k)]
        while heap:
            d, u = heappop(heap)
            if d != dist[u]:
                continue
            for v, w in adj[u]:
                if d + w < dist[v]:
                    dist[v] = d + w
                    heappush(heap, (dist[v], v))
        answer = max(dist[1:])
        return -1 if answer == float('inf') else answer
```

Listing 4: P02 — C++17

```
#include <algorithm>
#include <functional>
#include <queue>
#include <utility>
#include <vector>
using namespace std;
class Solution {
public:
    int networkDelayTime(vector<vector<int>>& times, int n, int k) {
        const int INF = 1000000000;
        vector<vector<pair<int, int>>> adj(n + 1);
        for (const auto& e : times)
            adj[e[0]].push_back({e[1], e[2]});
```

```
        vector<int> dist(n + 1, INF);
        using State = pair<int, int>;
        priority_queue<State, vector<State>, greater<State>> pq;
        dist[k] = 0;
        pq.push({0, k});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d != dist[u]) continue;
            for (auto [v, w] : adj[u]) {
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    pq.push({dist[v], v});
                }
            }
        }
        int ans = *max_element(dist.begin() + 1, dist.end());
        return ans == INF ? -1 : ans;
    }
};
```

3 LeetCode 1368 — Minimum Cost to Make at Least One Valid Path

Problem statement

Key idea: An edge *leaving* a cell costs zero if it follows that cell's arrow, otherwise one. Use 0–1 BFS. Arrow codes are 1 =right, 2 =left, 3 =down, 4 =up.

Correctness: Any simple grid path can be made valid by changing exactly those departure arrows it disobeys. Conversely a valid path in a modified grid uses at most the total number of changed cells. A minimum-cost graph walk can be made simple by removing non-negative-cost cycles, so it never demands two conflicting arrows at one cell. Thus the minimum graph cost equals the minimum number of modifications, and 0–1 BFS finds it.

Time and space: For RC cells there are at most $4RC$ edges: $O(RC)$ time and $O(RC)$ space for distances and the deque.

Pitfalls: Inspect the departure arrow, not the destination arrow. Do not permanently mark a cell on first insertion: a later zero-cost route may improve it. Arrows pointing outside simply have no corresponding in-grid zero edge.

Listing 5: P03 — Python 3

```
from collections import deque
from typing import List

class Solution:
    def minCost(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        moves = [(0, 1), (0, -1), (1, 0), (-1, 0)]
        dist = [[float('inf')] * cols for _ in range(rows)]
        dist[0][0] = 0
        queue = deque([(0, 0, 0)])
        while queue:
            d, r, c = queue.popleft()
            if d != dist[r][c]:
                continue
            for code, (dr, dc) in enumerate(moves, 1):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols:
                    w = int(grid[r][c] != code)
                    if d + w < dist[nr][nc]:
                        dist[nr][nc] = d + w
```

```
                        item = (d + w, nr, nc)
                        if w:
                            queue.append(item)
                        else:
                            queue.appendleft(item)
        return dist[-1][-1]
```

Listing 6: P03 — C++17

```
#include <array>
#include <deque>
#include <vector>
using namespace std;
class Solution {
public:
    int minCost(vector<vector<int>>& grid) {
        int R = grid.size(), C = grid[0].size();
        int dr[] = {0, 0, 1, -1}, dc[] = {1, -1, 0, 0};
        vector<vector<int>> dist(R, vector<int>(C, 1000000000));
        deque<array<int, 3>> dq;
        dist[0][0] = 0;
        dq.push_back({0, 0, 0});
        while (!dq.empty()) {
            auto [d, r, c] = dq.front(); dq.pop_front();
            if (d != dist[r][c]) continue;
            for (int k = 0; k < 4; ++k) {
                int nr = r + dr[k], nc = c + dc[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                int w = (grid[r][c] != k + 1);
                if (d + w < dist[nr][nc]) {
                    dist[nr][nc] = d + w;
                    if (w) dq.push_back({d + w, nr, nc});
                    else dq.push_front({d, nr, nc});
                }
            }
        }
        return dist[R - 1][C - 1];
    }
};
```

II. Core practice

4 LeetCode 1514 — Path with Maximum Probability

Problem statement

Key idea: Keep the largest known product at each vertex. A max-heap expands the most likely unfinished path. No logarithms are necessary.

Correctness: Every edge probability lies in $[0, 1]$, so extending a path never increases its product. Suppose a popped maximum label could be improved. The first unfinished vertex on the better path would have a label at least as large as that path's final product, contradicting the heap order. Thus a current pop finalises the optimal probability, including the destination's first current pop.

Time and space: For n vertices and m undirected edges: $O(n + m \log(m + 2))$ time and $O(n + m)$ space; lazy heap space is $O(m + 1)$.

Pitfalls: The graph is undirected; multiply probabilities, do not add them. Python negates heap keys. Use `double` in C++, and compare with a tolerance only in tests, not in relaxation logic. An unreachable destination has answer zero; the empty start-to-start path has probability one.

Listing 7: P04 — Python 3

```python
from heapq import heappop, heappush
from typing import List

class Solution:
    def maxProbability(self, n: int, edges: List[List[int]],
                       succProb: List[float], start_node: int,
                       end_node: int) -> float:
        adj = [[] for _ in range(n)]
        for (u, v), p in zip(edges, succProb):
            adj[u].append((v, p))
            adj[v].append((u, p))
        best = [0.0] * n
        best[start_node] = 1.0
        heap = [(-1.0, start_node)]
        while heap:
            negp, u = heappop(heap)
            p = -negp
            if p < best[u]:
                continue
            if u == end_node:
                return p
            for v, probability in adj[u]:
                candidate = p * probability
                if candidate > best[v]:
                    best[v] = candidate
                    heappush(heap, (-candidate, v))
        return 0.0
```

Listing 8: P04 — C++17

```cpp
#include <queue>
#include <utility>
#include <vector>
using namespace std;
class Solution {
public:
    double maxProbability(int n, vector<vector<int>>& edges,
                          vector<double>& succProb, int start_node,
                          int end_node) {
        vector<vector<pair<int, double>>> adj(n);
        for (int i = 0; i < (int)edges.size(); ++i) {
            int u = edges[i][0], v = edges[i][1];
            adj[u].push_back({v, succProb[i]});
            adj[v].push_back({u, succProb[i]});
        }
        vector<double> best(n, 0.0);
        priority_queue<pair<double, int>> pq;
        best[start_node] = 1.0;
        pq.push({1.0, start_node});
        while (!pq.empty()) {
            auto [p, u] = pq.top(); pq.pop();
            if (p < best[u]) continue;
            if (u == end_node) return p;
            for (auto [v, probability] : adj[u]) {
                double candidate = p * probability;
                if (candidate > best[v]) {
                    best[v] = candidate;
                    pq.push({candidate, v});
                }
            }
        }
        return 0.0;
    }
};
```

5 LeetCode 1631 — Path with Minimum Effort

Problem statement

Key idea: A path's effort is its largest edge difference. Replace additive relaxation by $\max(d[u], |h[u] - h[v]|)$.

Correctness: The label is the smallest known bottleneck of a real path. Extending a path cannot reduce its bottleneck, and the extension function is monotone in the old label. The usual first-unfinalised-vertex Dijkstra argument therefore applies with maximum instead of addition. The destination's final label is its minimum possible effort.

Time and space: With $N = RC$ cells and $O(N)$ edges: $O(N \log(N + 2))$ time, $O(N)$ space including the lazy heap.

Pitfalls: Do not sum height differences or compare every cell with the starting height. The empty path on a one-cell grid has effort zero. Marking visited at insertion is incorrect.

Listing 9: P05 — Python 3

```
from heapq import heappop, heappush
from typing import List

class Solution:
    def minimumEffortPath(self, heights: List[List[int]]) -> int:
        rows, cols = len(heights), len(heights[0])
        dist = [[float('inf')] * cols for _ in range(rows)]
        dist[0][0] = 0
        heap = [(0, 0, 0)]
        while heap:
            effort, r, c = heappop(heap)
            if effort != dist[r][c]:
                continue
            if (r, c) == (rows - 1, cols - 1):
                return effort
            for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols:
                    nd = max(effort, abs(heights[r][c] - heights[nr][nc]))
                    if nd < dist[nr][nc]:
                        dist[nr][nc] = nd
                        heappush(heap, (nd, nr, nc))
        return 0
```

Listing 10: P05 — C++17

```
#include <algorithm>
#include <array>
#include <cstdlib>
#include <functional>
#include <queue>
#include <vector>
using namespace std;
class Solution {
public:
    int minimumEffortPath(vector<vector<int>>& heights) {
        int R = heights.size(), C = heights[0].size();
        int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
        vector<vector<int>> dist(R, vector<int>(C, 1000000000));
        using State = array<int, 3>;
        priority_queue<State, vector<State>, greater<State>> pq;
        dist[0][0] = 0;
        pq.push({0, 0, 0});
        while (!pq.empty()) {
```

```
            auto [d, r, c] = pq.top(); pq.pop();
            if (d != dist[r][c]) continue;
            if (r == R - 1 && c == C - 1) return d;
            for (int k = 0; k < 4; ++k) {
                int nr = r + dr[k], nc = c + dc[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                int nd = max(d, abs(heights[r][c] - heights[nr][nc]));
                if (nd < dist[nr][nc]) {
                    dist[nr][nc] = nd;
                    pq.push({nd, nr, nc});
                }
            }
        }
        return 0;
    }
};
```

6 LeetCode 2290 — Minimum Obstacle Removal to Reach Corner

Problem statement

Key idea: Moving into (r, c) costs `grid[r][c]`. This is a 0–1 shortest path, not a shortest-step path.

Correctness: A simple path pays once for every obstacle cell it enters. Removing these obstacles makes that path usable. Conversely, any route after removals can be simplified, and its obstacle count is no larger than the number removed. Therefore minimum removals equal minimum edge cost; 0–1 BFS computes that cost.

Time and space: $O(RC)$ time and $O(RC)$ auxiliary space, including all deque entries.

Pitfalls: Charge the destination cell of a move. Both endpoints are guaranteed empty, so initialise the starting cost to zero. A cell first reached with many removals can later improve; keep distances rather than an insertion-time visited flag.

Listing 11: P06 — Python 3

```
from collections import deque
from typing import List

class Solution:
    def minimumObstacles(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        dist = [[float('inf')] * cols for _ in range(rows)]
        dist[0][0] = 0
        queue = deque([(0, 0, 0)])
        while queue:
            d, r, c = queue.popleft()
            if d != dist[r][c]:
                continue
            for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols:
                    w = grid[nr][nc]
                    if d + w < dist[nr][nc]:
                        dist[nr][nc] = d + w
                        item = (d + w, nr, nc)
                        if w:
                            queue.append(item)
                        else:
                            queue.appendleft(item)
        return dist[-1][-1]
```

Listing 12: P06 — C++17

```cpp
#include <array>
#include <deque>
#include <vector>
using namespace std;
class Solution {
public:
    int minimumObstacles(vector<vector<int>>& grid) {
        int R = grid.size(), C = grid[0].size();
        int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
        vector<vector<int>> dist(R, vector<int>(C, 1000000000));
        deque<array<int, 3>> dq;
        dist[0][0] = 0;
        dq.push_back({0, 0, 0});
        while (!dq.empty()) {
            auto [d, r, c] = dq.front(); dq.pop_front();
            if (d != dist[r][c]) continue;
            for (int k = 0; k < 4; ++k) {
                int nr = r + dr[k], nc = c + dc[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                int w = grid[nr][nc];
                if (d + w < dist[nr][nc]) {
                    dist[nr][nc] = d + w;
                    if (w) dq.push_back({d + w, nr, nc});
                    else dq.push_front({d, nr, nc});
                }
            }
        }
        return dist[R - 1][C - 1];
    }
};
```

7 LeetCode 542 — 01 Matrix

Problem statement

Key idea: Start BFS from *all* zero cells simultaneously. All four-neighbour edges have cost one, even when their destination contains zero.

Correctness: Imagine a super-source with zero-cost edges to every zero. Initialising every zero at distance zero implements this source without adding it. FIFO BFS visits cells in increasing distance layers, so first discovery gives the minimum distance from any zero, which is precisely the requested distance to a nearest zero in this undirected grid.

Time and space: Each cell is enqueued once and has at most four neighbours: $O(RC)$ time, $O(RC)$ space including output and queue (also $O(RC)$ queue space excluding output).

Pitfalls: This is not an obstacle-removal problem and not 0–1 BFS. Running a fresh BFS from every one cell is too slow. The statement guarantees at least one zero. Use -1 as undiscovered, since zero is a valid answer.

Listing 13: P07 — Python 3

```python
from collections import deque
from typing import List

class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        rows, cols = len(mat), len(mat[0])
        dist = [[-1] * cols for _ in range(rows)]
        queue = deque()
        for r in range(rows):
```

```
            for c in range(cols):
                if mat[r][c] == 0:
                    dist[r][c] = 0
                    queue.append((r, c))
        while queue:
            r, c = queue.popleft()
            for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                nr, nc = r + dr, c + dc
                if (0 <= nr < rows and 0 <= nc < cols
                        and dist[nr][nc] == -1):
                    dist[nr][nc] = dist[r][c] + 1
                    queue.append((nr, nc))
        return dist
```

Listing 14: P07 — C++17

```
#include <queue>
#include <utility>
#include <vector>
using namespace std;
class Solution {
public:
    vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {
        int R = mat.size(), C = mat[0].size();
        int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
        vector<vector<int>> dist(R, vector<int>(C, -1));
        queue<pair<int, int>> q;
        for (int r = 0; r < R; ++r)
            for (int c = 0; c < C; ++c)
                if (mat[r][c] == 0) {
                    dist[r][c] = 0;
                    q.push({r, c});
                }
        while (!q.empty()) {
            auto [r, c] = q.front(); q.pop();
            for (int k = 0; k < 4; ++k) {
                int nr = r + dr[k], nc = c + dc[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                if (dist[nr][nc] != -1) continue;
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
        return dist;
    }
};
```

8 AtCoder ABC176 D — Wizard in Maze

Official statement

Key idea: Walk to an orthogonally adjacent open cell for zero cost. Warp to any open cell with $|\Delta r| \le 2$ and $|\Delta c| \le 2$ for cost one, *including the four corners*. Use 0–1 BFS.

Correctness: Each graph edge is one allowed walk or warp, with cost equal to the number of warps it uses. Conversely every legal move appears in the graph. Thus paths and legal journeys have identical costs, and 0–1 BFS minimises the number of warps. Walls may be crossed by a warp, but its landing cell must be open.

Time and space: At most 24 warp offsets and four walking offsets per cell: $O(HW)$ time and $O(HW)$ space. The graph is generated on demand.

Pitfalls: Convert both input coordinates from 1-based indexing. Walls are never landing states. Diagonal walking is forbidden, diagonal warping is allowed. Output -1 if unreachable. Do not apply the corner exclusion of Problem 15 here.

Listing 15: P08 — Python 3

```
import sys
from collections import deque

def main():
    input = sys.stdin.buffer.readline
    h, w = map(int, input().split())
    sr, sc = (x - 1 for x in map(int, input().split()))
    tr, tc = (x - 1 for x in map(int, input().split()))
    grid = [input().strip() for _ in range(h)]
    inf = h * w + 1
    dist = [[inf] * w for _ in range(h)]
    dist[sr][sc] = 0
    queue = deque([(0, sr, sc)])
    walks = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    warps = [(dr, dc) for dr in range(-2, 3)
             for dc in range(-2, 3) if dr or dc]
    while queue:
        d, r, c = queue.popleft()
        if d != dist[r][c]:
            continue
        if (r, c) == (tr, tc):
            print(d)
            return
        for dr, dc in walks:
            nr, nc = r + dr, c + dc
            if (0 <= nr < h and 0 <= nc < w
                    and grid[nr][nc] == ord('.')
                    and d < dist[nr][nc]):
                dist[nr][nc] = d
                queue.appendleft((d, nr, nc))
        for dr, dc in warps:
            nr, nc = r + dr, c + dc
            if (0 <= nr < h and 0 <= nc < w
                    and grid[nr][nc] == ord('.')
                    and d + 1 < dist[nr][nc]):
                dist[nr][nc] = d + 1
                queue.append((d + 1, nr, nc))
    print(-1)

if __name__ == '__main__':
    main()
```

Listing 16: P08 — C++17

```
#include <array>
#include <deque>
#include <iostream>
#include <string>
#include <vector>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int H, W, sr, sc, tr, tc;
    cin >> H >> W >> sr >> sc >> tr >> tc;
    --sr; --sc; --tr; --tc;
    vector<string> grid(H);
    for (auto& row : grid) cin >> row;
    const int INF = H * W + 1;
    vector<vector<int>> dist(H, vector<int>(W, INF));
```

```
    deque<array<int, 3>> dq;
    int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
    dist[sr][sc] = 0;
    dq.push_back({0, sr, sc});
    while (!dq.empty()) {
        auto [d, r, c] = dq.front(); dq.pop_front();
        if (d != dist[r][c]) continue;
        if (r == tr && c == tc) { cout << d << '\n'; return 0; }
        for (int k = 0; k < 4; ++k) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= H || nc < 0 || nc >= W) continue;
            if (grid[nr][nc] == '.' && d < dist[nr][nc]) {
                dist[nr][nc] = d;
                dq.push_front({d, nr, nc});
            }
        }
        for (int a = -2; a <= 2; ++a)
            for (int b = -2; b <= 2; ++b) {
                int nr = r + a, nc = c + b;
                if (nr < 0 || nr >= H || nc < 0 || nc >= W) continue;
                if (grid[nr][nc] == '.' && d + 1 < dist[nr][nc]) {
                    dist[nr][nc] = d + 1;
                    dq.push_back({d + 1, nr, nc});
                }
            }
    }
    cout << -1 << '\n';
}
```

9 Project Euler 81 — Path Sum: Two Ways

Original problem and matrix link

Key idea: Moving right or down into a cell costs its value. Initialise the top-left distance to the *top-left value*, not zero.

Correctness: Every admissible path corresponds to right/down graph edges. Its initial value plus its edge costs counts every visited cell exactly once, including both endpoints. Thus Dijkstra from the top-left to bottom-right yields the minimum sum. The graph is a DAG, so the alternative recurrence $D[r, c] = A[r, c] + \min(D[r-1, c], D[r, c-1])$ also works, with boundary cases handled separately.

Time and space: The supplied shared Dijkstra implementation takes $O(RC \log(RC+2))$ time and $O(RC)$ space including its lazy heap and matrix. A specialised row-by-row DP can achieve $O(RC)$ time and $O(C)$ working space excluding the input matrix; it is not the implementation below.

Pitfalls: No left or up moves. Count the starting cell. The programs below read CSV rows without a leading dimension, not the displayed mathematical matrix syntax.

10 Project Euler 82 — Path Sum: Three Ways

Original problem and matrix link

Key idea: This is *any* left-column cell to *any* right-column cell, with up, down and right moves. Initialise every left-column cell to its own value.

Correctness: A virtual source with an edge of weight $A[r, 0]$ to each left-column cell exactly models the choice of starting row and its cost. All subsequent edges charge their destination. The first current heap pop in the rightmost column is the cheapest path to any allowed endpoint, by Dijkstra's finalisation property.

Time and space: The supplied implementation takes $O(RC \log(RC+2))$ time and $O(RC)$ space, including $O(RC)$ heap entries; the R initial sources do not change this bound.

Pitfalls: Do not force either endpoint to a corner. Up moves matter; left moves are forbidden. One-column input returns the smallest cell value, not the sum of the column.

11 Project Euler 83 — Path Sum: Four Ways

Original problem and matrix link

Key idea: Use all four orthogonal directions; source and target are the top-left and bottom-right cells. Destination-entry costs plus the starting value again give the path sum.

Correctness: The same cost-preserving correspondence as Problem 9 holds with four directions. Positive cell values mean deleting a cycle strictly reduces its cost, so a shortest walk can be taken to be simple. All edge costs are non-negative; Dijkstra therefore returns the requested minimum sum. A right/down-only DP no longer enumerates all admissible paths.

Time and space: $O(RC \log(RC+2))$ time and $O(RC)$ space for the matrix, labels and lazy heap.

Pitfalls: Do not reuse the move set or multi-source initialisation of Problem 10. Use 64-bit sums in C++ even though the official answer is smaller. No downloading or network access is performed by either program.

Complete shared programs for Problems 9–11

Save either listing as a standalone program. For example, run `python3 euler.py 81 < matrix.csv`, or compile with `c++ -std=c++17 -O2 euler.cpp -o euler` and run `./euler 82 < matrix.csv`. Choose `81`, `82` or `83` as the sole argument. Input is a nonempty rectangular matrix of positive integers, one comma-separated row per line, with no row/column counts. Blank lines are ignored. The official instances are square 80×80 matrices; download the linked data separately if desired, then supply it on stdin.

The official 5×5 example CSV is:

```
131,673,234,103,18
201,96,342,965,150
630,803,746,422,111
537,699,497,121,956
805,732,524,37,331
```

The three modes must print **2427**, **994** and **2297**, respectively. The full official data answers are 427337 (81), 260324 (82) and 425185 (83); these are checksums for the respective input files, not constants embedded in the algorithms. Both listings were also checked on a data-only mirror of the published matrices, since direct official downloads were unavailable during validation.

Listing 17: P09–P11 — Python 3 (choose mode on the command line)

```
import csv
import sys
from heapq import heappop, heappush

def solve(matrix, mode):
    rows, cols = len(matrix), len(matrix[0])
    if mode == 81:
        moves = [(1, 0), (0, 1)]
    elif mode == 82:
        moves = [(-1, 0), (1, 0), (0, 1)]
    else:
        moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
    dist = [[float('inf')] * cols for _ in range(rows)]
    heap = []
    sources = range(rows) if mode == 82 else [0]
    for r in sources:
        dist[r][0] = matrix[r][0]
        heappush(heap, (matrix[r][0], r, 0))
    while heap:
        d, r, c = heappop(heap)
        if d != dist[r][c]:
            continue
        if c == cols - 1 and (mode == 82 or r == rows - 1):
            return d
        for dr, dc in moves:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                nd = d + matrix[nr][nc]
                if nd < dist[nr][nc]:
                    dist[nr][nc] = nd
                    heappush(heap, (nd, nr, nc))
    raise ValueError('No path')

def main():
    if len(sys.argv) != 2 or sys.argv[1] not in {'81', '82', '83'}:
        raise SystemExit('Usage: python3 euler.py 81|82|83 < matrix.csv')
    matrix = [[int(x) for x in row] for row in csv.reader(sys.stdin)
              if row and any(x.strip() for x in row)]
    if (not matrix or not matrix[0]
            or any(len(row) != len(matrix[0]) for row in matrix)
            or any(x <= 0 for row in matrix for x in row)):
        raise SystemExit('Expected a rectangular positive-integer CSV matrix')
    print(solve(matrix, int(sys.argv[1])))

if __name__ == '__main__':
    main()
```

Listing 18: P09–P11 — C++17 (choose mode on the command line)

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <queue>
#include <sstream>
#include <string>
#include <tuple>
#include <utility>
#include <vector>
using namespace std;
int main(int argc, char** argv) {
    if (argc != 2 || (string(argv[1]) != "81" &&
        string(argv[1]) != "82" && string(argv[1]) != "83")) {
        cerr << "Usage: ./euler 81|82|83 < matrix.csv\n";
        return 1;
    }
    int mode = stoi(argv[1]);
    using ll = long long;
    vector<vector<ll>> a;
    string line;
    while (getline(cin, line)) {
        if (line.find_first_not_of(" \t\r") == string::npos) continue;
        replace(line.begin(), line.end(), ',', ' ');
        istringstream in(line);
        vector<ll> row;
        ll x;
        while (in >> x) row.push_back(x);
        if (row.empty() || (!a.empty() && row.size() != a[0].size())) {
```

```
            cerr << "Expected a rectangular CSV matrix\n";
            return 1;
        }
        for (ll value : row) if (value <= 0) {
            cerr << "Expected positive cell values\n";
            return 1;
        }
        a.push_back(row);
    }
    if (a.empty()) return 1;
    int R = a.size(), C = a[0].size();
    vector<pair<int, int>> moves = {{1, 0}, {0, 1}};
    if (mode == 82) moves = {{-1, 0}, {1, 0}, {0, 1}};
    if (mode == 83) moves = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
    const ll INF = (1LL << 62);
    vector<vector<ll>> dist(R, vector<ll>(C, INF));
    using State = tuple<ll, int, int>;
    priority_queue<State, vector<State>, greater<State>> pq;
    for (int r = 0; r < (mode == 82 ? R : 1); ++r) {
        dist[r][0] = a[r][0];
        pq.push({a[r][0], r, 0});
    }
    while (!pq.empty()) {
        auto [d, r, c] = pq.top(); pq.pop();
        if (d != dist[r][c]) continue;
        if (c == C - 1 && (mode == 82 || r == R - 1)) {
            cout << d << '\n';
            return 0;
        }
        for (auto [dr, dc] : moves) {
            int nr = r + dr, nc = c + dc;
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
            ll nd = d + a[nr][nc];
            if (nd < dist[nr][nc]) {
                dist[nr][nc] = nd;
                pq.push({nd, nr, nc});
            }
        }
    }
}
```

III. Extended homework — C++17

12 LeetCode 1928 — Minimum Cost to Reach Destination in Time

Problem statement

Key idea: One cheapest label per city loses important information: a dearer but faster arrival can be necessary. Keep *exact elapsed time* as part of the state. Since every edge travel time is positive, time orders the states as a DAG.

Let $D[t, v]$ be the cheapest fee total to reach city v using exactly t minutes. Initialise $D[0, 0] = \text{fee}[0]$ and all other states to infinity. For each time t and undirected edge (u, v, τ) with $t \geq \tau$, relax

$$D[t, v] \leftarrow \min(D[t, v], D[t - \tau, u] + \text{fee}[v])$$

and the symmetric transition. Return $\min_{0 \leq t \leq T} D[t, n - 1]$, or -1 if all are infinite.

Correctness: Induct on exact time t. The base is the empty route with its starting fee. Every nonempty route has a last edge of positive duration τ, so its prefix is in an earlier, already correct time layer. The recurrence considers that prefix and charges the entered city's fee; conversely every

recurrence candidate describes a legal route. Thus every exact-time value is optimal. Minimising all allowed arrival times gives precisely the budget-constrained optimum.

Time and space: For time budget T, n cities and m input edges: $O(T(n+m))$ time including table initialisation and $O(Tn)$ auxiliary space, excluding the supplied edge list. No heap is used. This fits the official $T \leq 1000$ constraints.

Pitfalls: Charge the source fee and each entered city's fee, including the destination. Edges are undirected and parallel edges can have different travel times; do not overwrite one with an arbitrary other edge. Do not return the minimum-fee route if it exceeds the time budget. The recurrence relies on strictly positive travel times.

Listing 19: P12 — C++17

```
#include <algorithm>
#include <vector>
using namespace std;
class Solution {
public:
    int minCost(int maxTime, vector<vector<int>>& edges,
                vector<int>& passingFees) {
        int n = passingFees.size();
        const int INF = 1000000000;
        vector<vector<int>> dp(maxTime + 1, vector<int>(n, INF));
        dp[0][0] = passingFees[0];
        for (int t = 1; t <= maxTime; ++t) {
            for (const auto& e : edges) {
                int u = e[0], v = e[1], travel = e[2];
                if (travel > t) continue;
                if (dp[t - travel][u] != INF)
                    dp[t][v] = min(dp[t][v],
                        dp[t - travel][u] + passingFees[v]);
                if (dp[t - travel][v] != INF)
                    dp[t][u] = min(dp[t][u],
                        dp[t - travel][v] + passingFees[u]);
            }
        }
        int ans = INF;
        for (int t = 0; t <= maxTime; ++t)
            ans = min(ans, dp[t][n - 1]);
        return ans == INF ? -1 : ans;
    }
};
```

13 Codeforces 786B — Legacy

Official statement

Key idea: Compress a range of possible endpoints with a segment tree, but use *two directed trees* to avoid unintended free travel between original vertices.

The n original vertices are shared leaves. For every internal interval create an *out-tree* node, whose zero-cost edges point to its two children, and an *in-tree* node, whose zero-cost edges point from its two children to the parent. Thus an out-node reaches exactly its interval's leaves; exactly those leaves can reach an in-node.

- Type 1, $v \to u$ of weight w: insert that edge directly.
- Type 2, $v \to [l, r]$: decompose $[l, r]$ into $O(\log n)$ segment nodes and add $v \to \text{out}[p]$ of weight w for each.
- Type 3, $[l, r] \to v$: add $\text{in}[p] \to v$ of weight w for those same covering nodes.

Then run Dijkstra from s and print only original vertices.

Correctness: Every permitted portal is represented by one weighted query edge and an appropriate zero-cost descent or ascent, so every original journey has the same-cost compressed walk. Conversely a zero-cost out-tree descent can reach only a leaf in its interval, and an in-tree ascent can start only in that interval. The two trees are not joined at internal nodes: they meet only at original leaves. Splitting any compressed walk at its original vertices therefore gives legal query transitions of identical total cost. The distance problems are equivalent, and non-negative-weight Dijkstra solves the compressed one.

Time and space: There are $N = 3n - 2$ vertices (or one for $n = 1$), $O(n)$ tree edges and $O(q \log(n+1))$ query edges. Put $M = O(n + q \log(n+1))$. Construction takes $O(M)$ time; Dijkstra takes $O(N + M \log(M+2))$ time. Total space is $O(N + M)$, including an $O(M+1)$ lazy heap. Recursion depth is $O(\log(n+1))$, not $O(n)$.

Pitfalls: Direction is crucial; an undirected segment tree introduces illegal free portals. Input is 1-based; convert vertices and interval endpoints consistently. Distances can exceed 32 bits. Do not expand each range into individual edges, which can create $\Theta(nq)$ edges.

Listing 20: P13 — C++17

```
#include <functional>
#include <iostream>
#include <queue>
#include <utility>
#include <vector>
using namespace std;
using ll = long long;
struct SegmentGraph {
    vector<vector<pair<int, ll>>> g;
    vector<int> out, in;
    SegmentGraph(int n) : g(n), out(4 * n), in(4 * n) {
        build(1, 0, n - 1);
    }
    int node() {
        g.emplace_back();
        return (int)g.size() - 1;
    }
    void edge(int u, int v, ll w) { g[u].push_back({v, w}); }
    void build(int p, int l, int r) {
        if (l == r) { out[p] = in[p] = l; return; }
        out[p] = node(); in[p] = node();
        int mid = (l + r) / 2;
        build(p * 2, l, mid);
        build(p * 2 + 1, mid + 1, r);
        for (int child : {p * 2, p * 2 + 1}) {
            edge(out[p], out[child], 0);
            edge(in[child], in[p], 0);
        }
    }
    void range(int p, int l, int r, int ql, int qr,
               int v, ll w, int type) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) {
            if (type == 2) edge(v, out[p], w);
            else edge(in[p], v, w);
            return;
        }
        int mid = (l + r) / 2;
        range(p * 2, l, mid, ql, qr, v, w, type);
        range(p * 2 + 1, mid + 1, r, ql, qr, v, w, type);
    }
};
int main() {
```

```
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q, s;
    cin >> n >> q >> s; --s;
    SegmentGraph graph(n);
    for (int i = 0; i < q; ++i) {
        int type, v; cin >> type >> v; --v;
        ll w;
        if (type == 1) {
            int u; cin >> u >> w; --u;
            graph.edge(v, u, w);
        } else {
            int l, r; cin >> l >> r >> w; --l; --r;
            graph.range(1, 0, n - 1, l, r, v, w, type);
        }
    }
    const ll INF = (1LL << 62);
    vector<ll> dist(graph.g.size(), INF);
    using State = pair<ll, int>;
    priority_queue<State, vector<State>, greater<State>> pq;
    dist[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue;
        for (auto [v, w] : graph.g[u]) {
            if (d + w < dist[v]) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    for (int v = 0; v < n; ++v)
        cout << (dist[v] == INF ? -1 : dist[v])
             << (v + 1 == n ? '\n' : ' ');
}
```

14 Codeforces 715B — Complete The Graph

Official statement

Key idea: Find the *first missing edge whose activation crosses the target distance.* Only that pivot needs a carefully chosen increase. Binary search makes this repeated-Dijkstra construction fast enough and easy to prove.

Let the missing edges, in input order, be $e_1, \ldots, e_K$, and let $B = L + 1$. In configuration k, give $e_1, \ldots, e_k$ weight 1, and all later missing edges weight B. Fixed edges never change. Let $f(k)$ be the shortest s–t distance, or infinity. It is nonincreasing in k.

1. If $f(0) < L$, a fixed-edge path is already too short: return NO. If $f(0) = L$, output configuration 0.
2. If $f(K) > L$, even all minimum allowed weights are too long: return NO.
3. Otherwise binary-search the smallest k with $f(k) \le L$. Put $D = f(k)$ and replace the pivot weight 1 by $1 + L - D$ in configuration k. Output it (the code also verifies its final shortest distance).

Correctness: Since $B > L$, a path of length at most L cannot use a disabled edge. Thus $f(0) < L$ is an immutable fixed-only obstruction; $f(K)$ is a lower bound for all assignments. In the remaining case, $k \ge 1$ exists and $f(k-1) > L$. Every path not using pivot e_k is unchanged between configurations $k-1$ and k, hence has length greater than L. Every shortest path of length $D \le L$ in configuration k must therefore use the pivot. Positive weights allow us to consider only simple paths, which use that undirected edge exactly once. Increasing its weight by $L - D$ makes every

pivot-using simple path have length at least $D + (L - D) = L$, while a previously shortest one has length exactly L. Nonpivot paths remain too long. Therefore the final shortest distance is exactly L. All assigned weights lie in $[1, L + 1] \subseteq [1, 10^{18}]$.

Time and space: Binary search uses $O(\log(K + 2))$ Dijkstra runs, each $O(n + m \log(m + 2))$ including configuration/reset costs. Total time is $O((n + m \log(m + 2)) \log(K + 2))$, and space $O(n + m)$, including the $O(m + 1)$ lazy heap. This handles $n \leq 1000$, $m \leq 10000$ without one run per missing edge.

Pitfalls: Vertices are 0-based. A zero input weight means *unknown*, not a traversable zero-cost edge. Store each undirected edge once and reference its ID from both endpoints, so both directions always use the same assigned weight. Do not simply add the entire deficit to an arbitrary unknown edge on an arbitrary path: competing paths can invalidate that construction. Use 64-bit distances and preserve all known weights.

Listing 21: P14 — C++17

```
#include <functional>
#include <iostream>
#include <queue>
#include <utility>
#include <vector>
using namespace std;
using ll = long long;
struct Edge { int u, v; ll w; };
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, s, t;
    ll L;
    cin >> n >> m >> L >> s >> t;
    vector<Edge> edges(m);
    vector<vector<pair<int, int>>> adj(n);
    vector<int> missing;
    for (int i = 0; i < m; ++i) {
        cin >> edges[i].u >> edges[i].v >> edges[i].w;
        adj[edges[i].u].push_back({edges[i].v, i});
        adj[edges[i].v].push_back({edges[i].u, i});
        if (edges[i].w == 0) missing.push_back(i);
    }
    const ll BIG = L + 1, INF = (1LL << 62);
    auto configure = [&](int k) {
        for (int j = 0; j < (int)missing.size(); ++j)
            edges[missing[j]].w = (j < k ? 1 : BIG);
    };
    auto shortest = [&]() -> ll {
        vector<ll> dist(n, INF);
        using State = pair<ll, int>;
        priority_queue<State, vector<State>, greater<State>> pq;
        dist[s] = 0; pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d != dist[u]) continue;
            if (u == t) return d;
            for (auto [v, id] : adj[u]) {
                ll nd = d + edges[id].w;
                if (nd < dist[v]) {
                    dist[v] = nd;
                    pq.push({nd, v});
                }
            }
        }
        return INF;
    };
    configure(0);
```

```
    ll d0 = shortest();
    if (d0 < L) { cout << "NO\n"; return 0; }
    if (d0 > L) {
        int K = missing.size();
        configure(K);
        if (shortest() > L) { cout << "NO\n"; return 0; }
        int lo = 1, hi = K;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            configure(mid);
            if (shortest() <= L) hi = mid;
            else lo = mid + 1;
        }
        configure(lo);
        ll D = shortest();
        edges[missing[lo - 1]].w += L - D;
    }
    if (shortest() != L) { cout << "NO\n"; return 0; }
    cout << "YES\n";
    for (const auto& e : edges)
        cout << e.u << ' ' << e.v << ' ' << e.w << '\n';
}
```

15 AtCoder ABC213 E — Stronger Takahashi

Official statement

Key idea: Model the useful reach of a punch, not all cells in a 5×5 square. A free edge goes to an orthogonally adjacent cell only if that destination is originally open. A paid edge may land on either an open cell or a wall, using offsets

$$-2 \le \Delta r, \Delta c \le 2, \qquad (\Delta r, \Delta c) \neq (0,0), \qquad \neg\big(|\Delta r| = 2 \ \wedge \ |\Delta c| = 2\big).$$

In particular, the four offsets $(\pm 2, \pm 2)$ are **excluded**. All other nonzero offsets in the square are allowed for one punch, even when the landing cell is a wall. Run 0–1 BFS.

Why these offsets? One punch clears a 2×2 square. If that square contains a cell orthogonally adjacent to our current cell, we can enter it and reach any of its four cells. The union of these reachable cells over all such squares is exactly the offset set above. At a boundary, every included in-grid destination still has a suitable in-grid 2×2 square because $H, W \ge 2$. A diagonal $(2,2)$ cell cannot be reached this way in one punch through an otherwise solid block region.

Correctness: Each zero edge is an actual free walk, and each paid edge can be realised by punching an appropriate adjacent 2×2 square and walking inside it. Hence a graph path of cost p gives a journey with at most p punches. For the reverse direction, take any successful set of p punched squares. Ignore squares disconnected from the starting reachable region, and order the useful squares so each touches or overlaps the region made reachable by earlier squares and original open cells; such an order exists by following connectivity from the start in the cleared grid. Before square i, inductively every reached cell has graph distance at most $i - 1$. From an adjacent reached cell, every cell of the new square has a paid graph edge of the allowed shape, so has distance at most i. Original open-cell walks extend this bound for free. If a square was already reachable it adds nothing. After at most p squares the destination therefore has graph distance at most p. The two inequalities prove equivalence, without storing which walls were previously destroyed. 0–1 BFS then gives the optimum.

Time and space: At most 20 paid offsets plus four free neighbours per cell, generated on demand: $O(HW)$ time and $O(HW)$ space including deque entries.

Pitfalls: Unlike ABC176D, paid landings may be walls and the four far corners are forbidden. Unlike obstacle removal, one punch clears a whole 2×2 square, not one cell. A zero-cost edge may

enter only an originally open cell; treating an arbitrary wall state as freely enterable is invalid. A paid jump to an already open cell is still allowed and can cross intervening walls.

Listing 22: P15 — C++17

```
#include <array>
#include <cstdlib>
#include <deque>
#include <iostream>
#include <string>
#include <vector>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int H, W; cin >> H >> W;
    vector<string> grid(H);
    for (auto& row : grid) cin >> row;
    vector<vector<int>> dist(H, vector<int>(W, H * W + 1));
    deque<array<int, 3>> dq;
    dist[0][0] = 0;
    dq.push_back({0, 0, 0});
    while (!dq.empty()) {
        auto [d, r, c] = dq.front(); dq.pop_front();
        if (d != dist[r][c]) continue;
        if (r == H - 1 && c == W - 1) {
            cout << d << '\n'; return 0;
        }
        for (int dr = -2; dr <= 2; ++dr)
            for (int dc = -2; dc <= 2; ++dc) {
                if ((dr == 0 && dc == 0) ||
                    (abs(dr) == 2 && abs(dc) == 2)) continue;
                int nr = r + dr, nc = c + dc;
                if (nr < 0 || nr >= H || nc < 0 || nc >= W) continue;
                int cost = !(abs(dr) + abs(dc) == 1 &&
                             grid[nr][nc] == '.');
                if (d + cost < dist[nr][nc]) {
                    dist[nr][nc] = d + cost;
                    if (cost) dq.push_back({d + 1, nr, nc});
                    else dq.push_front({d, nr, nc});
                }
            }
    }
}
```